

Introduction to Computers and Programming

CSCI 1101 Introduction to Programming

Topics

- Introduction
- Hardware and Software
- How Computers Store Data
- How a Program Works
- Using Python

Introduction

Computers can be programmed

- Designed to do any job that a program tells them to

Program: set of instructions that a computer follows to perform a task

- Commonly referred to as *Software*

Programmer: person who can design, create, and test computer programs

- Also known as software developer

Hardware and Software

Hardware: The physical devices that make up a computer

- Computer is a system composed of several components that all work together

Typical major components:

- Central processing unit (CPU)
- Main memory (RAM)
- Secondary storage devices
- Input and output devices



CPU

Central processing unit (CPU): the part of the computer that actually runs programs

- Most important component
- Without it, cannot run software



Main Memory

Main memory known as *Random Access Memory (RAM)*: where computer stores a program while program is running, and data used by the program

- CPU can quickly access data in RAM
- Volatile memory used for temporary storage while program is running
- Contents are erased when computer is off



Secondary Storage Devices

Secondary storage can hold data for long periods of time

- Programs normally stored here and loaded to main memory when needed

Types of secondary memory:

- **Hard disk drive (HDD):** magnetically encodes data onto a spinning circular disk
- **Solid state drive (SSD):** faster than disk drive, no moving parts, stores data in solid state memory
- Flash memory: portable, no physical disk



Input Devices

Input data the computer collects from people and other devices

- Examples: keyboard, mouse, touchscreen, scanner, camera



Output Devices

Output data produced by the computer for other people or devices



Multimedia Projector



Monitor



Printer

Output Devices



Plotter



Micro Films

Software (1 of 2)

Everything the computer does is controlled by software

- Application software
- System software

Application software: programs that make computer useful for everyday tasks

- Examples: word processing, email, games, and Web browsers



Software (2 of 2)

System software: programs that control and manage basic operations of a computer

- **Operating system (OS):** controls operations of software and hardware components
- **Utility Program:** performs specific task to enhance computer operation or safeguard data
- **Basic input/output system (BIOS):** firmware interface that initializes hardware components.



How Computers Store Data

All data in a computer is stored in sequences of 0s and 1s known as binary or machine language.

Byte: just enough memory to store letter or small number

- Divided into eight bits
- **Bit:** electrical component that can hold positive or negative charge, like on/off switch
- The on/off pattern of bits in a byte represents data stored in the byte

How a Program Works (1 of 3)

CPU designed to perform simple operations on pieces of data

- Examples: reading data, adding, subtracting, multiplying, and dividing numbers
- Understands instructions written in machine language and included in its instruction set

To carry out meaningful calculation, CPU must perform many operations

How a Program Works (2 of 3)

Program must be copied from secondary memory to RAM each time CPU executes it

CPU executes program in cycle:

- **Fetch:** read the next instruction from memory into CPU
- **Decode:** CPU decodes fetched instruction to determine which operation to perform
- **Execute:** perform the operation

How a Program Works (3 of 3)

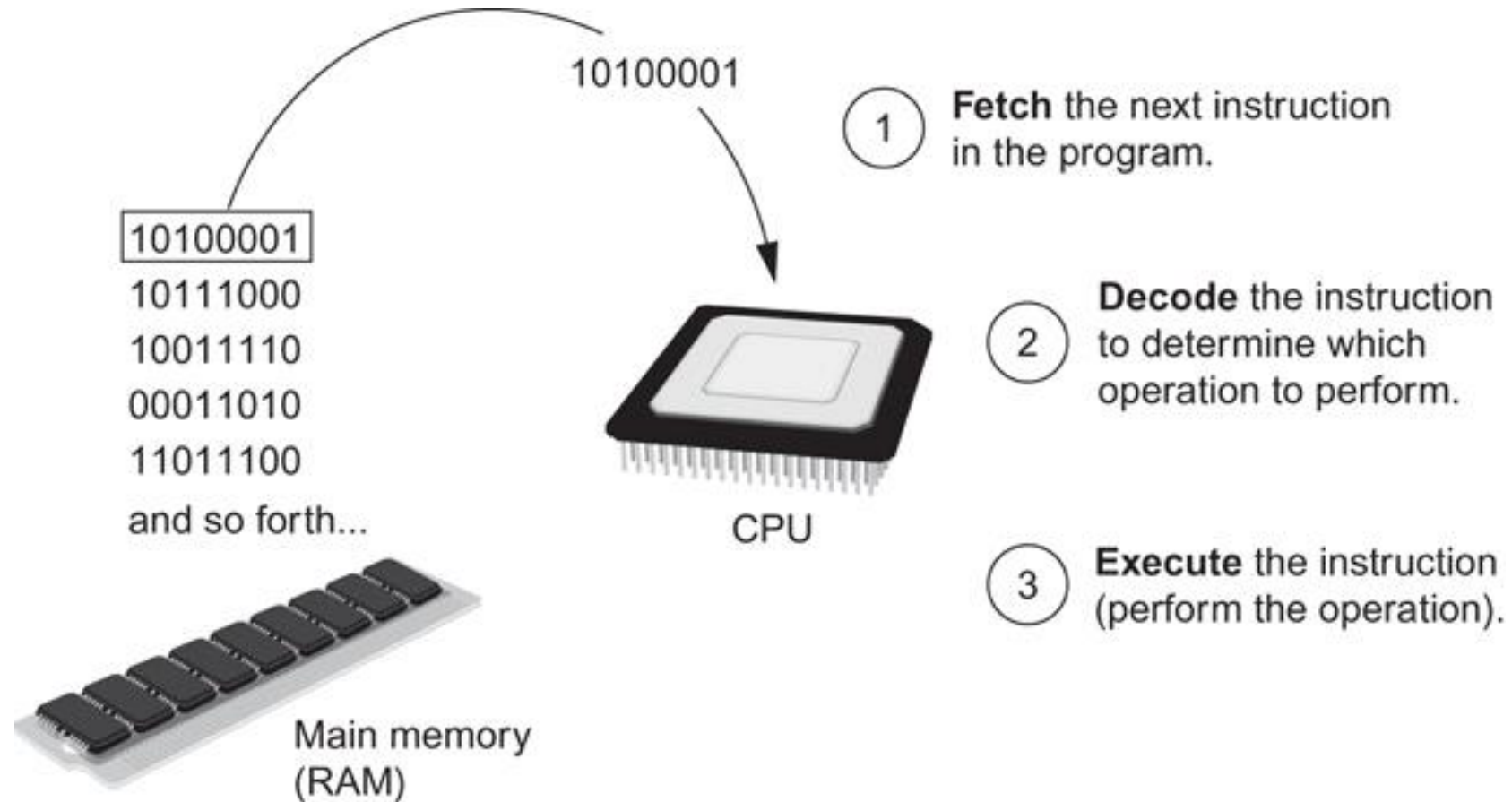


Figure 1-16 The fetch-decode-execute cycle

Machine Language to Assembly Language

Impractical for people to write in machine language

Assembly language: uses short words (mnemonics) for instructions instead of binary numbers

- Easier for programmers to work with

Assembler: translates assembly language to machine language for execution by CPU

High-Level Languages

Low-level language: close in nature to machine language

- Example: assembly language

High-Level language: allows simple creation of powerful and complex programs

- No need to know how CPU works or write large number of instructions
- More intuitive to understand
- Example: Python

Keywords, Operators, and Syntax

Keywords: predefined words used to write program in high-level language

- Each keyword has specific meaning

Operators: perform operations on data

- Example: math operators to perform arithmetic

Syntax: set of rules to be followed when writing program

Statement: individual instruction used in high-level language

Compilers and Interpreters (1 of 3)

Programs written in high-level languages must be translated into machine language to be executed

Compiler: translates high-level language program into separate machine language program

- Machine language program can be executed at any time

Compilers and Interpreters (2 of 3)

Interpreter: translates and executes instructions in high-level language program

- Used by Python language
- Interprets one instruction at a time
- No separate machine language program

Source code: statements written by programmer

- **Syntax error:** prevents code from being translated

Compilers and Interpreters (3 of 3)

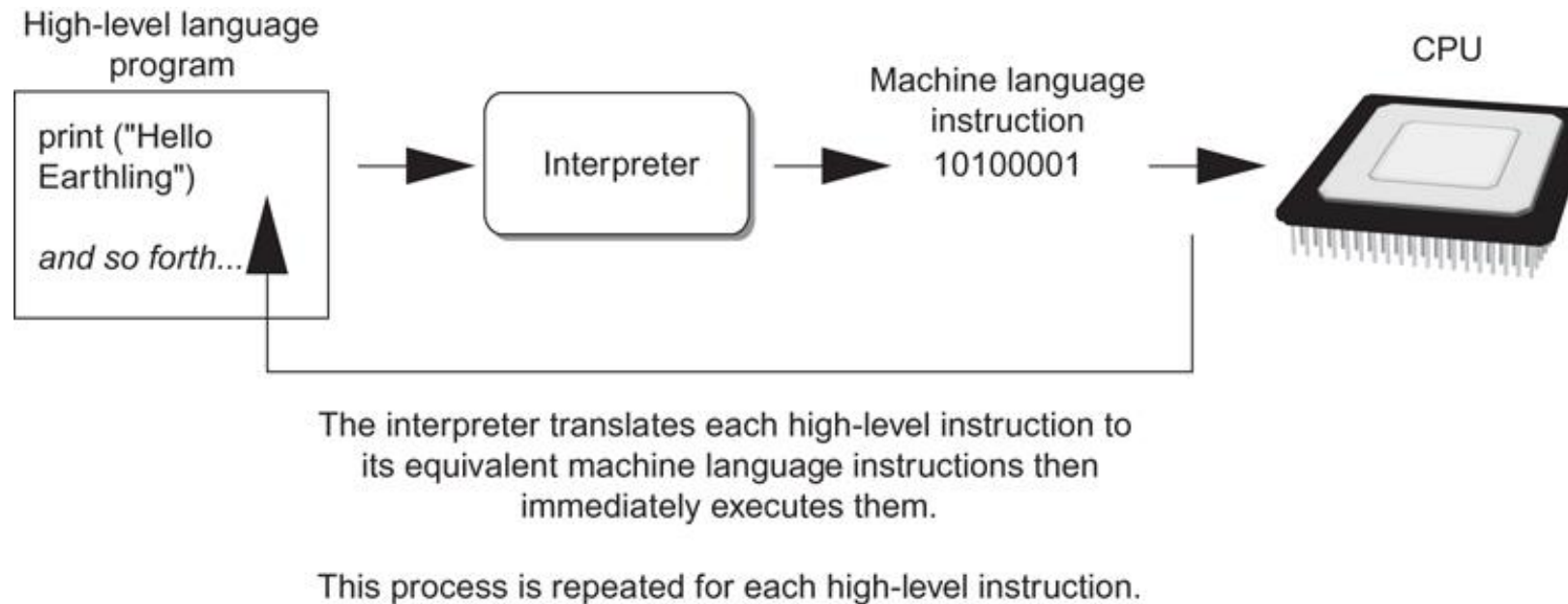


Figure 1-19 Executing a high-level program with an interpreter

The IDE Programming Environment

IDE (Integrated Development Program): single program that provides tools to write, execute and test a program

- Automatically installed when Python language is installed
- Runs in interactive mode
- Has built-in text editor with features designed to help write Python programs